K.K Wagh Institute of Engineering Education and Research, Nashik

Department of Computer Engineering

PBL PROJECT REPORT

Project Based Learning (PBL)

Course Code: 2301310
Course Title: Project Based Learning
Academic Year: 2026–27
Semester: V
Class: T.Y. B.Tech. Computer Engineering
Division: A

PID: 3

PROJECT TITLE

"CIVICORBIT: AN AI-POWERED GAMIFIED PLATFORM FOR CIVIC ENGAGEMENT TRACKING"

(Working prototype: CivicMitra)

Submitted by

Roll No.	Name of Student	CRN	Signature
36	Omkar Kahandal		
46	Sanchita Misar		
47	Abhijith Nair		
49	Harsh Navandar		

Under the Guidance of

Faculty/Mentor Name:

Department of Computer Engineering
K.K Wagh Institute of Engineering Education and Research, Nashik

Academic Year 2026–27

CERTIFICATE

This is to certify that the Mini Project titled "**CivicOrbit: An AI-Powered Gamified Platform for Civic Engagement Tracking**" has been successfully completed by the above-mentioned student team as part of the requirements of Project Based Learning (Course Code: 2301310) during Academic Year 2026–27.

The project work has been carried out under the guidance of the faculty mentor and demonstrates the application of problem identification, collaborative learning, system design, implementation, testing, documentation, presentation and reflection.

Faculty/Mentor Signature: _____

Date: _____

STUDENT DECLARATION

We hereby declare that the project work presented in this report is the work carried out by our team as part of the Project Based Learning course. The work has been undertaken collaboratively and the sources, tools, datasets and references used have been appropriately acknowledged.

Student Signatures:

1. Omkar Kahandal _____
2. Sanchita Misar _____
3. Abhijith Nair _____
4. Harsh Navandar _____

ACKNOWLEDGEMENT

We would like to thank our faculty mentor for the guidance, regular feedback and encouragement given to us throughout this project. We are grateful to the Head of the Department of Computer Engineering and to K.K Wagh Institute of Engineering Education and Research, Nashik, for giving us the opportunity and the facilities to carry out this work under the Project Based Learning course.

We also acknowledge the open-source community behind React, Vite, Tailwind CSS and the other libraries we used, and the Google Firebase and Google Gemini platforms, whose free tools made it possible to build and test a working prototype. Finally, we thank our classmates and friends who tried the application and gave us useful suggestions.

ABSTRACT

Civic participation in India is often invisible: citizens vote, file complaints on municipal portals, sign petitions, volunteer, and donate, but there is no unified system that recognises, measures, or rewards this behaviour, leaving genuine engagement under-incentivised and difficult to sustain. CivicOrbit addresses this gap as a cross-platform mobile application that converts everyday civic actions into verifiable "Civic Points" using an on-device AI detection pipeline. Users share screenshots or proof of civic activity (voter confirmations, complaint receipts, petition signatures, volunteering sign-ups, donation receipts, or event RSVPs) through a share-sheet or accessibility-based capture flow; the app performs on-device pre-filtering and OCR (ML Kit / Core ML) before an optional cloud verification stage confirms source authenticity, checks for duplicates through perceptual hashing and text similarity, and assigns points under a tiered trust model (Low, Medium, High verification). Points map to a solar-system-themed progression system, from Mercury to the legacy Pluto tier, complemented by streak-based sub-badges, a real-time leaderboard (global, city, and organisation scoped, built on Firebase/WebSocket updates), and shareable achievement cards for WhatsApp and Instagram. The system is designed privacy-first: screen content is processed on-device wherever possible, raw evidence is retained only for a short verification window, and users retain full transparency and control over what has been scanned. Anti-fraud safeguards including rate limiting, category caps, device fingerprinting, and a moderator review queue protect the integrity of the leaderboard. Built on a Flutter/React Native client with a Node.js/Python microservice backend (PostgreSQL, Redis, S3), CivicOrbit demonstrates how gamification and applied AI can be combined responsibly to encourage sustained civic participation while remaining compliant with Google Play, Apple App Store, and data-protection regulations such as India's DPDP Act.

Keywords: Civic Engagement, Gamification, On-device AI, OCR, Screen-Scan Detection, Points and Leveling System, Real-time Leaderboard, Anti-fraud Verification, Privacy-first Design, Mobile Application

TABLE OF CONTENTS

1. Introduction
2. Problem Identification and Background Study
3. Problem Definition and Requirements
4. Project Planning and Gantt Chart
5. System Design and Architecture
6. Module Identification and Technology Selection
7. Implementation
8. Testing and Validation
9. Results and Discussion
10. Project Demonstration
11. Ethical, Social and Sustainability Considerations
12. Teamwork and Individual Contributions
13. Challenges Faced and Solutions Adopted
14. Learning Outcomes and Reflection
15. Conclusion and Future Scope
16. References
17. Appendices

1. INTRODUCTION

1.1 Background

Civic engagement means taking part in the life of one's community. This includes voting, reporting local problems to the municipal body, signing petitions, volunteering, donating, and adopting everyday habits that protect shared resources such as water, energy and green spaces. Most of this effort happens quietly. A person who files a complaint or saves water at home receives no record of it and no recognition, so it is easy to lose motivation.

At the same time, gamification (using points, levels, streaks and leaderboards outside of games) has been shown to increase motivation in many apps, and modern multimodal AI models can now look at a photo or screenshot and judge what it shows. CivicOrbit combines these two ideas into one platform.

1.2 Motivation

We noticed that people around us regularly do small civic and eco-friendly things but never keep track of them and rarely share them. We wanted to build something simple that makes this effort visible and a little bit fun, and that also lets us apply what we have learned in web development, cloud databases and AI APIs in a real project.

1.3 Need for the Project

- There is no single place where different kinds of civic and eco actions are recorded, measured and rewarded.
- Reward systems that depend on self-reporting can be cheated, so a verification step is needed.
- Users need to see the real effect of their actions (for example, litres of water or kilograms of CO_2 saved) to stay motivated.
- A social layer (feed, followers, leaderboard) helps people encourage each other to continue.

1.4 Scope of the Project

The abstract of this report describes the complete CivicOrbit vision as a cross-platform mobile application. Within the time of this PBL course, we built a working web prototype of the core idea, named CivicMitra in the user interface. The scope of the prototype is as follows.

Included

- User registration and login (email/password and Google)
- Daily and general challenges in seven categories with points and difficulty levels
- Proof submission by photo or screenshot, verified by the Gemini AI model
- AI Screen Scanner with screen grab, camera and file upload options
- Points, ten-level progression, streaks, badges and an environmental impact summary
- Community feed, follow system, leaderboard, events and daily quiz
- Admin pages to manage challenges and quiz questions

Excluded

- Native Android/iOS application and on-device OCR (ML Kit / Core ML)

- Perceptual hashing and text-similarity matching (the prototype uses an exact SHA-256 hash)
- Solar-system level names (Mercury to Pluto); the prototype uses eco-themed level names
- WhatsApp / Instagram achievement cards, device fingerprinting and a PostgreSQL/Redis/S3 backend
- Integration with real government portals

1.5 Project Objectives

Objective 1: Design and develop a web platform where users can complete civic/eco challenges and submit photo or screenshot proof.

Objective 2: Integrate an AI model that automatically verifies the submitted proof and blocks duplicate or irrelevant submissions.

Objective 3: Implement a points, level, streak and badge system together with an impact summary (CO_2, electricity, water and waste saved).

Objective 4: Provide community features (feed, follow, leaderboard, events) and keep user data secure with authentication and database rules.

2. PROBLEM IDENTIFICATION AND BACKGROUND STUDY

2.1 Problem Identification

Citizens carry out many civic actions, but these actions are scattered across different portals, apps and offline activities. There is no common system that collects them, checks that they are genuine, and gives the citizen feedback or recognition. As a result, participation depends only on personal willingness and is hard to sustain.

2.2 Existing System / Existing Solutions

- Municipal and government grievance portals and apps (for example, the Swachhata app and MyGov), which accept complaints but do not reward or track a citizen's overall participation.
- Online petition and volunteering platforms, which handle only one type of action each.
- General habit-tracking and eco apps, which are mostly based on self-reported entries.

2.3 Limitations of Existing Solutions

- Each platform covers a single activity, so there is no combined view of a person's civic contribution.
- Little or no motivation system such as points, levels, streaks or leaderboards.
- Self-reported actions are not verified, which makes any reward system easy to cheat.
- The real-world effect of the action is not shown to the user.

2.4 Background / Literature Study

Deterding et al. [1] define gamification as the use of game design elements in non-game contexts, and Hamari et al. [2] reviewed empirical studies and found that gamification generally has positive effects on engagement, although the results depend on the context and on the users. This supported our decision to use points, levels, streaks and badges.

Multimodal models such as Gemini [3] can accept an image together with a text instruction and return a structured answer. The Gemini API documentation [4] describes how to request the output in JSON format, which we use to get a verified/not-verified decision, a confidence score and a short reason. For the application itself we studied the Firebase documentation [5] for authentication, Firestore and security rules, and the React and Vite documentation [6]. For privacy requirements we referred to India's Digital Personal Data Protection Act, 2023 [7].

2.5 Problem Analysis

We broke the problem into four questions: (a) what actions should count, (b) how can an action be proved, (c) how can fake or repeated proof be stopped, and (d) how can users be kept interested. For (a) we chose seven categories of challenges (water, energy, transport, waste, food, community and nature). For (b) we chose photo or screenshot proof. For (c) we decided on AI verification, a hash-based duplicate check and daily scan limits. For (d) we added points, levels, streaks, badges, a feed and a leaderboard. We also decided to show the impact in real units, because numbers like "360 L of water saved" are easier to relate to than points.

2.6 SDG / Social Relevance

Relevant SDG: SDG 11 – Sustainable Cities and Communities; SDG 12 – Responsible Consumption and Production; SDG 16 – Peace, Justice and Strong Institutions, SDG 17: Partnerships for the Goals

Justification: The platform encourages citizens to take part in community life (SDG 11 and 16) and to adopt habits that save water, energy and reduce waste and emissions and Partnerships for the Goals (connecting citizens, NGOs, and civic organizations) (SDG 12 and 17).

3. PROBLEM DEFINITION AND REQUIREMENTS

3.1 Problem Statement

To design and develop an AI-assisted, gamified platform that lets citizens record everyday civic and eco-friendly actions with photo or screenshot proof, automatically verifies that proof, rewards verified actions with points, levels and badges, and shows the resulting impact and community ranking, while protecting user data and preventing misuse.

3.2 Functional Requirements

Sr. No.	Functional Requirement	Description
1	User authentication	Register and log in with email/password or Google; role-based access for User and Admin.
2	Challenge listing	Show daily and general challenges with category, difficulty, points and search/filter.
3	Proof submission	Submit a photo or screenshot as proof for a selected challenge.
4	AI verification	Send the proof to the Gemini model and receive verified/not verified, a score and a reason.
5	AI Screen Scanner	Capture proof by screen grab, camera or upload; match it automatically to an active challenge.
6	Points and levels	Award points for verified proofs; map total points to one of ten levels.
7	Streaks and badges	Track daily streaks and unlock badges for points, quizzes, events and proofs.
8	Impact summary	Convert points to CO_2, electricity, water and waste saved.
9	Community feed	Post updates with photos, follow users and view global/following feeds.
10	Leaderboard	Rank users by points (top 50).
11	Events and quiz	Register for eco events and take a daily 5-question quiz for points.
12	Admin management	Admin can add/edit challenges and quiz questions.

3.3 Non-Functional Requirements

Sr. No.	Requirement	Description
1	Performance	AI response within the 15-second limit per model call; pages load quickly using a Vite build.
2	Security	Firebase Authentication; Firestore rules allow users to modify only their own data; the Gemini API key stays on the server.
3	Usability	Clean, responsive interface with sidebar navigation, light/dark theme and clear feedback messages.
4	Scalability	Serverless API functions and managed Firebase services scale with the number of users.
5	Reliability	If the first AI model fails or times out, a second model is tried automatically.
6	Privacy	Only a single frame is processed per scan; users can switch the scanner off in Settings.

3.4 Users / Stakeholders

Stakeholder	Requirement / Role
Citizen / User	Complete challenges, submit proof, earn rewards, view impact and rank.
Admin / Moderator	Create and manage challenges and quiz questions; monitor platform activity.
Event organisers / NGOs	Publish eco events that users can register for.
Municipal bodies / institutions (future)	Use aggregated participation data to understand community involvement.

4. PROJECT PLANNING AND GANTT CHART

4.1 Project Team Formation

Roll No.	Student Name	Assigned Role
36	Omkar Kahandal	Documentation and community features
46	Sanchita Misar	Frontend design, dashboard and challenges
47	Abhijith Nair	Database design, authentication and security rules, Backend and AI verification API
49	Harsh Navandar	Testing

4.2 Work Breakdown Structure

Task	Responsible Member	Start	End	Status
Problem Identification	All members	Week 1	Week 1	Completed
Literature Study	Abhijith, Harsh	Week 1	Week 2	Completed
Requirement Analysis	Sanchita, Harsh	Week 2	Week 3	Completed
System Design	Omkar, Abhijith	Week 3	Week 4	Completed
Implementation	All members	Week 4	Week 8	Completed
Testing	Harsh, Omkar	Week 7	Week 8	Completed
Documentation	All members	Week 8	Week 9	Completed
Demonstration	All members	Week 9	Week 9	Completed

4.3 Gantt Chart

Task	W1	W2	W3	W4	W5	W6	W7	W8	W9
Problem Identification	■								
Literature Study	■	■							
Requirement Analysis		■	■						
System Design			■	■					
Implementation				■	■	■	■	■	
Testing							■	■	
Documentation								■	■
Demonstration									■

Figure 4.1: Project Gantt chart (9 weeks). Shaded cells show the weeks in which each task was carried out.

5. SYSTEM DESIGN AND ARCHITECTURE

5.1 Proposed System

The proposed system is a web application in which a user logs in, picks a challenge and submits a photo or screenshot as proof. The proof is sent to a small server-side API, which passes it to the Gemini model along with the challenge instructions. The model returns whether the proof is valid and how confident it is. If the proof is valid, the application awards points, updates the streak, level, badges and impact figures, and the activity can be shared on the community feed and counted on the leaderboard. The AI Screen Scanner offers the same idea in a faster way: the user captures the screen, camera or a file and the AI itself finds which active challenge it matches.

5.2 System Architecture

The system has three parts: a React client, a thin server layer (serverless functions on Vercel, or an Express server during development) that talks to Gemini, and Firebase for authentication and data storage. The Gemini key is kept on the server so that it is never exposed to the browser.

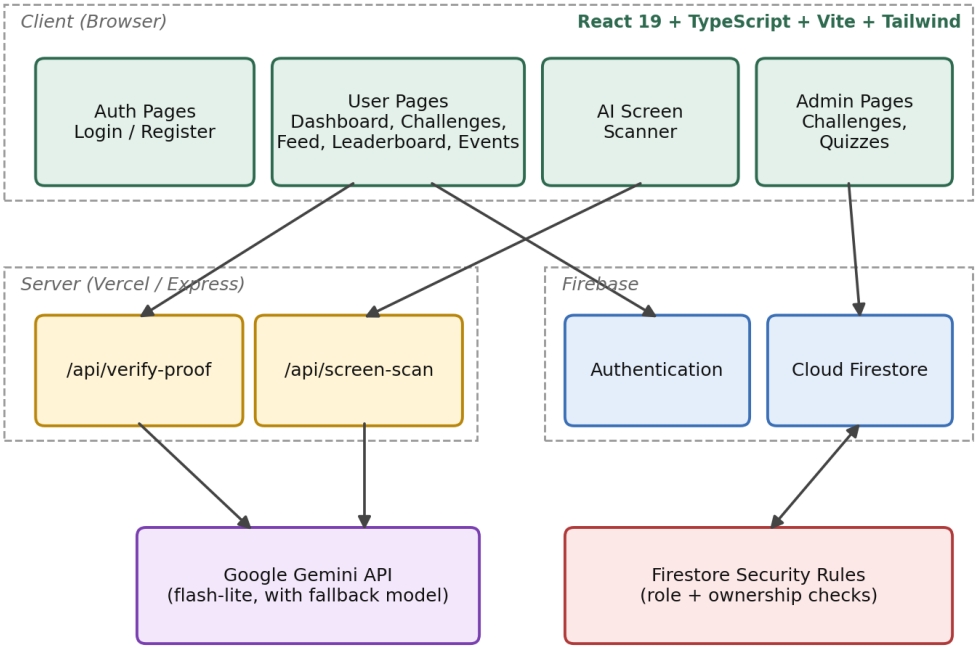


Figure 5.1: System architecture of CivicMitra

5.3 Data Flow / Workflow

Figure 5.2 shows what happens when a user submits a proof through the scanner. Limits and duplicate checks are done before the AI is called, so that no AI time is spent on invalid requests.

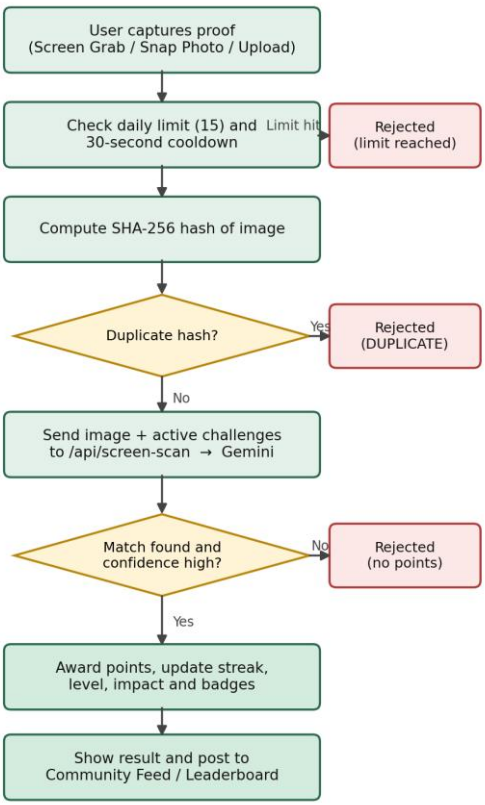


Figure 5.2: Proof verification workflow

5.4 Use Case Diagram

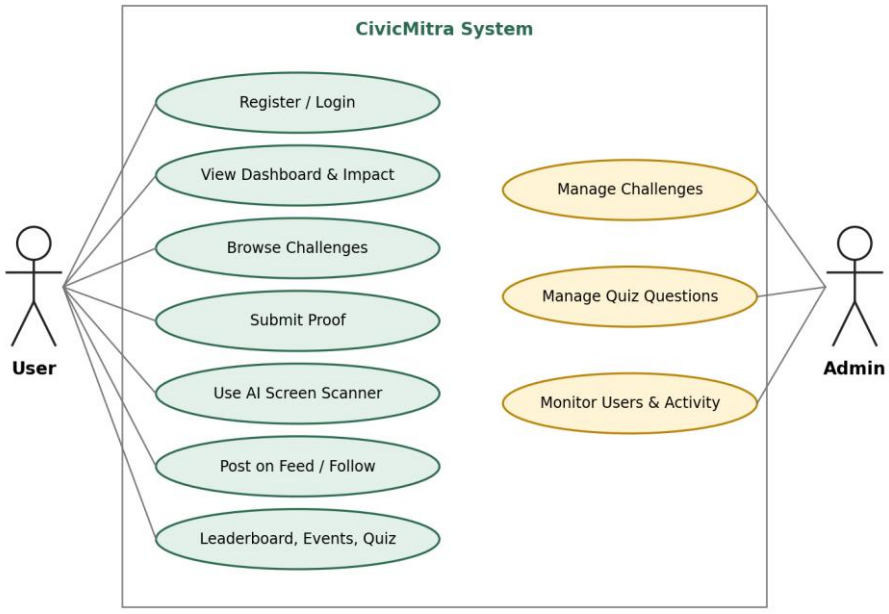


Figure 5.3: Use case diagram

5.5 Database / Data Design

Data is stored in Cloud Firestore as collections: users, challenges, completions, screenScans, badges, notifications, events with event_registrations, and quiz_questions with quiz_attempts. Likes and comments are stored under each completion. A separate usernames collection keeps usernames unique. Figure 5.4 shows the main collections and their relationships.

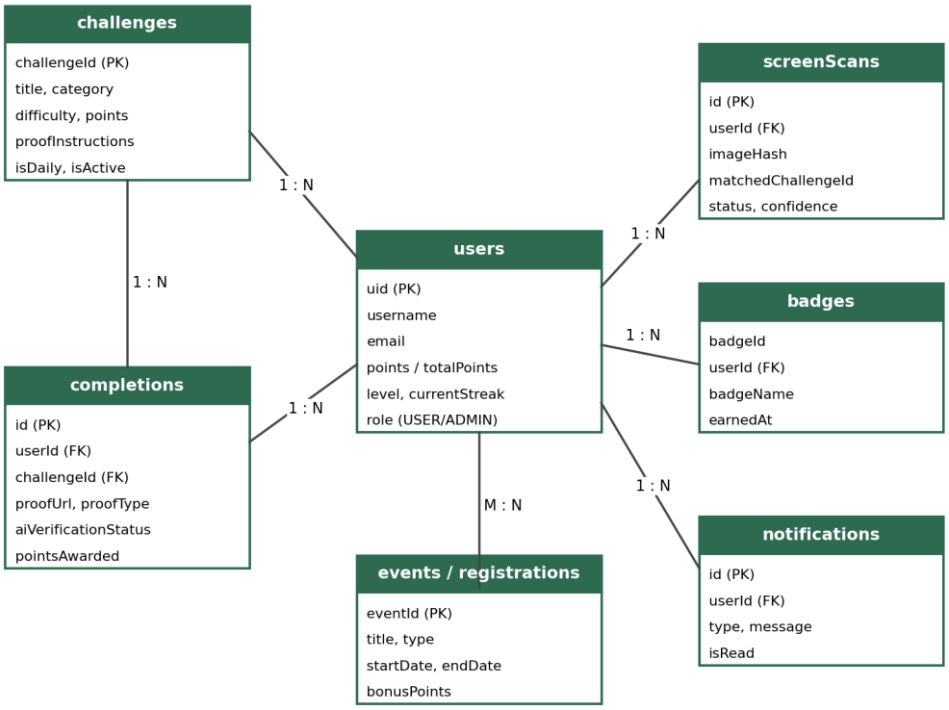


Figure 5.4: ER diagram of the main collections

5.6 Module Description

Module	Description	Responsible Member
M1 – Authentication and Profile	Registration, login (email and Google), role handling, profile and settings.	Abhijith
M2 – Challenges and Proof Submission	Challenge catalogue, filters, challenge details and proof upload.	Sanchita
M3 – AI Verification and Screen Scanner	Server APIs that call Gemini, scan limits, duplicate detection.	Abhijith
M4 – Gamification and Impact	Points, levels, streaks, badges, impact calculation, dashboard.	Sanchita, Omkar
M5 – Community	Feed, follow system, likes/comments, leaderboard.	Harsh
M6 – Events, Quiz and Admin	Eco events, daily quiz, admin panels for challenges and quizzes.	Abhijith

6. MODULE IDENTIFICATION AND TECHNOLOGY SELECTION

6.1 Technology Stack

Component	Technology / Tool	Reason for Selection
Front End	React 19, TypeScript, Tailwind CSS, Recharts	Component-based UI, type safety, fast styling, ready-made charts.
Back End	Node.js with Express (development) and Vercel serverless functions	Simple REST endpoints; the same code can be deployed without managing a server.
Database	Cloud Firestore	Real-time updates for feed and leaderboard; no database server to manage.
Authentication	Firebase Authentication	Secure email/password and Google sign-in out of the box.
AI	Google Gemini API (@google/genai)	Accepts image + text and returns JSON; free tier suitable for a prototype.
Framework / Build	Vite, React Router	Fast builds and client-side routing.
Development Tool	Google AI Studio, VS Code, Git/GitHub	Quick prototyping, editing and version control.
Testing Tool	TypeScript compiler (tsc), browser DevTools, manual testing	Catches type errors early; verifies behaviour in the browser.

6.2 Technology Selection Justification

We chose a web stack because it can be built, tested and demonstrated quickly without installing anything on the user's phone, and the same code can later be wrapped as a mobile app. React with TypeScript gave us reusable components and fewer runtime errors. Firebase removed the need to write our own login system and database server, and its security rules let us control who can read or write each document. Gemini was selected because a single API call can both read the image and give a structured decision, which avoids building and training a separate image-classification model. Vercel hosting lets the frontend and the API functions be deployed together.

6.3 Prototype

The prototype follows the architecture in Figure 5.1. Figure 6.1 shows the main dashboard of the working prototype, with the user's streak, points, level, environmental impact, daily challenges and the AI Scanner button.

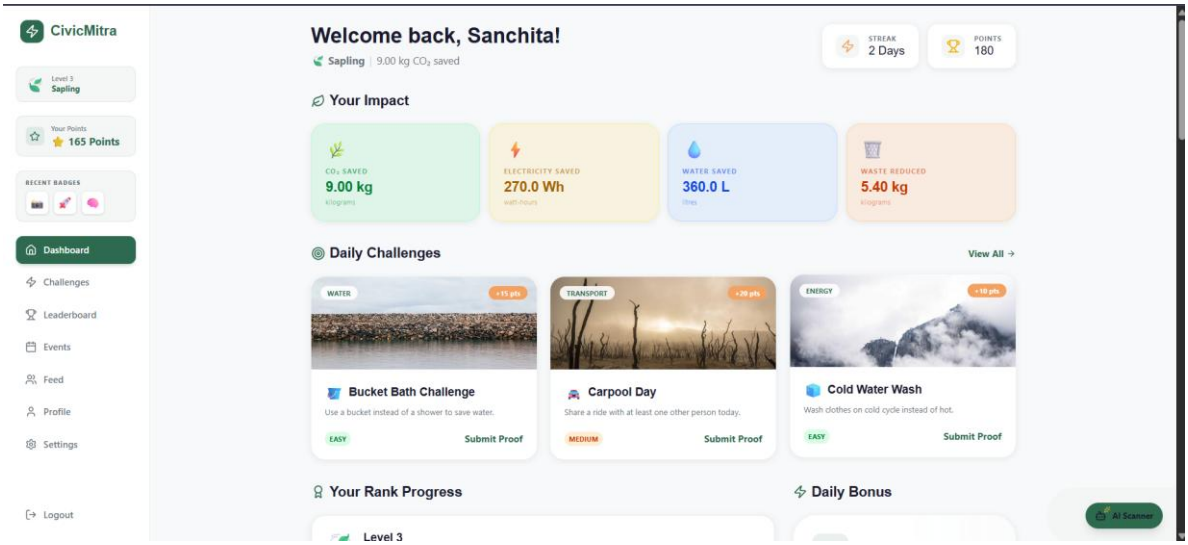


Figure 6.1: Dashboard of the CivicMitra prototype

7. IMPLEMENTATION

7.1 Development Environment

- Hardware: personal laptops/desktops of the team members.
- Software: Node.js and npm, Visual Studio Code, Git and GitHub, a Chromium-based browser.
- Services: Firebase project (Authentication and Firestore), Google Gemini API key, Vercel for hosting.
- Run locally with npm install and npm run dev; the GEMINI_API_KEY is set in an environment file.

7.2 Module-wise Implementation

Module 1: Authentication and Profile

Description: Lets users create an account, log in, and manage their profile and settings.

Implementation: Firebase Authentication handles email/password and Google sign-in. After sign-up, a user document and a unique username entry are created in Firestore. An AuthContext keeps track of the logged-in user, and protected routes send unauthenticated visitors to the login page. Admin access is decided by the role stored for the user. The Settings page contains account, notification, privacy, appearance and AI Scanner options.

Module 2: Challenges and Proof Submission

Description: Shows the available challenges and lets the user submit proof for one of them.

Implementation: Challenges are read from the challenges collection and displayed as cards with category, points and difficulty, with a search box and category filter (Figure 7.1). Clicking a card opens a modal that shows the instructions and the proof required. After the user uploads a photo, the image is compressed in the browser and sent to the /api/verify-proof endpoint. If the AI verifies it, a completion record is saved and the user sees the result in Figure 7.2.

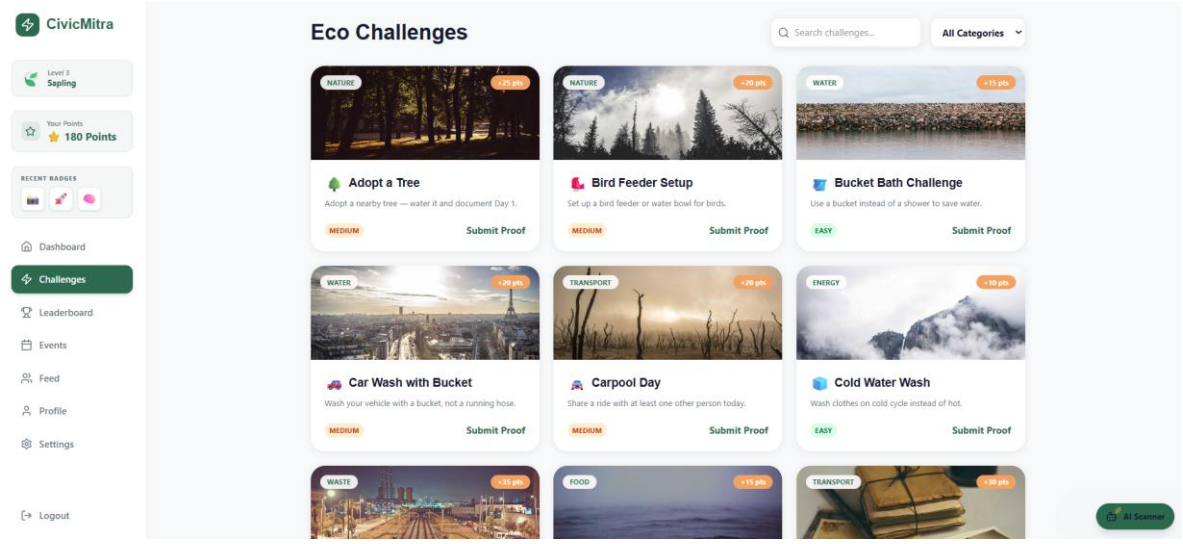


Figure 7.1: Eco Challenges page with search and category filter

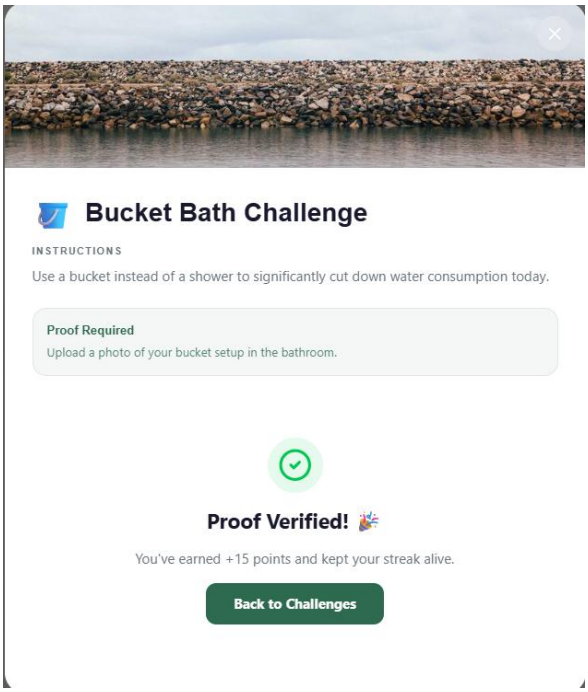


Figure 7.2: Challenge modal showing "Proof Verified" and points earned

Module 3: AI Verification and Screen Scanner

Description: Verifies proof with AI and provides a quick way to capture proof from the screen, camera or a file.

Implementation: The scanner (Figure 7.3) offers three options: Screen Grab, Snap Photo and Upload Proof. Only one frame is captured. The client checks the daily limit and cooldown, computes a SHA-256 hash and compares it with earlier scans to reject duplicates, and then calls /api/screen-scan with the image and the list of active challenges. The server builds a prompt, calls Gemini with a JSON response format, and returns the matched challenge, confidence and reason. Each scan is saved in the screenScans collection with the status VERIFIED, REJECTED or DUPLICATE.

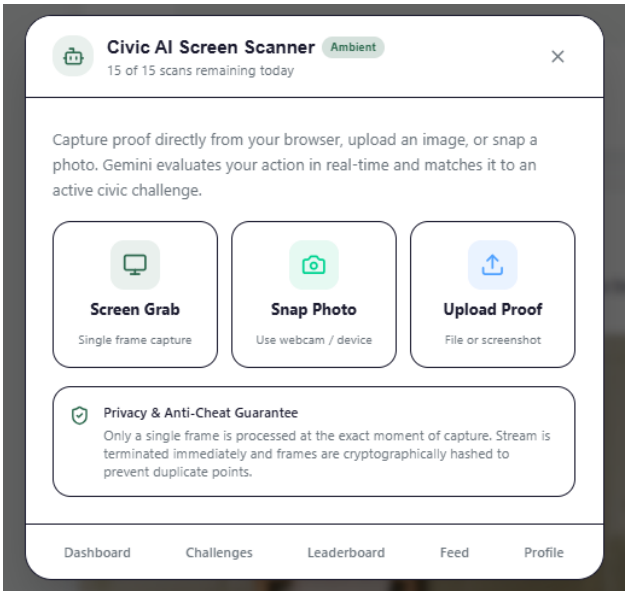


Figure 7.3: Civic AI Screen Scanner

Module 4: Gamification and Impact

Description: Turns verified actions into points, levels, streaks, badges and impact numbers.

Implementation: Points are added to the user's record; the level is calculated from the total (Table 7.1); the streak increases when the user completes a challenge on consecutive days. Badges are checked after each activity and unlocked when their condition is satisfied, for example "First Proof" or "Quiz Taker". The dashboard (Figure 6.1) converts points into CO_2, electricity, water and waste using fixed factors described in Section 7.3.

Module 5: Community

Description: Lets users share progress and see how others are doing.

Implementation: The Feed page shows posts from everyone (Global) or from followed users (Following). Users can write a post, attach a photo, and verified challenge completions appear with a "Verified" tag (Figure 7.4). Likes, comments and a follow button are supported. The Leaderboard page lists the top 50 users ordered by points, using a live Firestore query.

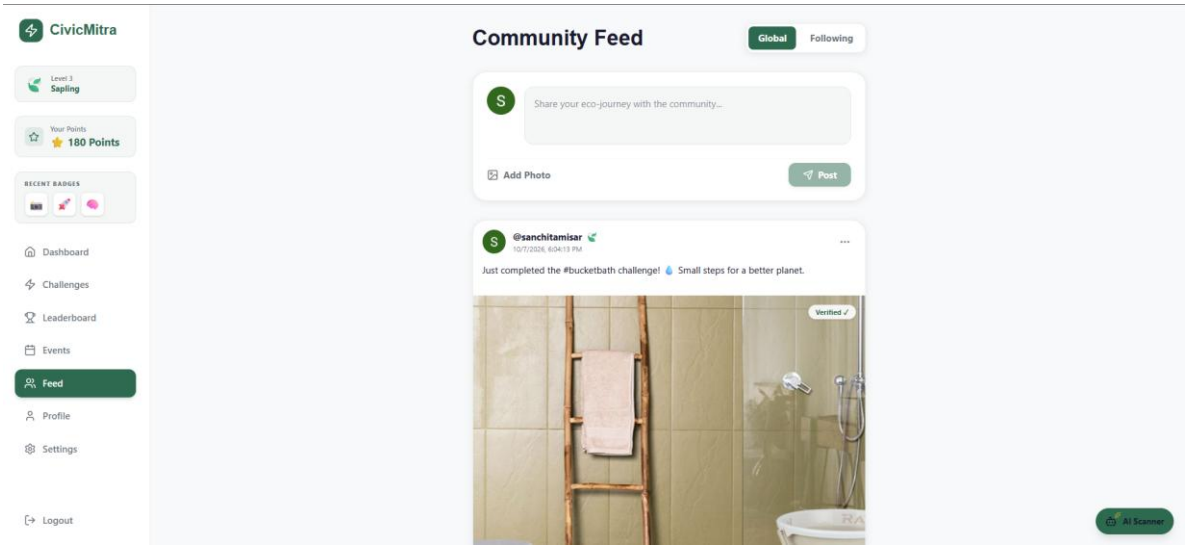


Figure 7.4: Community Feed showing a verified challenge post

Module 6: Events, Quiz and Admin

Description: Adds events, a daily quiz and management tools for the admin.

Implementation: The Events page lists eco events such as walks, clean-ups and plantations with bonus points; users can register and later submit proof. The daily quiz has five questions worth 10 points each. Admin pages let an administrator add, edit or disable challenges and manage quiz questions.

7.3 Important Algorithms / Methods

(a) Level calculation

The level is found by checking which range the user's total points fall into. Progress to the next level is (points − current minimum) ÷ (next minimum − current minimum) × 100.

Level	Title	Points range
1	Seedling	0 – 49
2	Sprout	50 – 99

Level	Title	Points range
3	Sapling	100 – 199
4	Grower	200 – 349
5	Tree Guardian	350 – 499
6	Eco Warrior	500 – 699
7	Earth Defender	700 – 999
8	Green Champion	1000 – 1499
9	Sustainability Hero	1500 – 1999
10	Climate Legend	2000 and above

Table 7.1: Level thresholds

(b) Impact conversion

Impact is estimated from points using fixed factors: CO_2 saved = points × 0.05 kg; electricity saved = points × 1.5 Wh; water saved = points × 2.0 L; waste reduced = points × 0.03 kg. For example, 180 points give 9.00 kg CO_2, 270 Wh, 360 L and 5.40 kg, which matches the dashboard in Figure 6.1. These factors are approximations chosen for the prototype.

(c) Duplicate detection

The captured image is converted to bytes and hashed with SHA-256 using the browser's Web Crypto API. If the same hash already exists for the user in the screenScans collection, the scan is marked DUPLICATE and no points are given.

(d) Rate limiting

Each user can make at most 15 scans per day and must wait 30 seconds between scans. The counters are kept per user and per date in the browser.

(e) AI verification with timeout and fallback

The server calls the first Gemini model and waits at most 15 seconds, using a timeout race. If it fails or times out, the second model is tried. The prompt asks the model to return JSON with verified, score (0 to 1) and reason, so the result can be used directly by the application. Images are also compressed to a maximum of 1200 px before upload to keep the request small.

7.4 Source Code Repository

Repository Link: https://github.com/Project-Based-Learning-Project/CivicMitra

Branch/Version: main

Live Prototype: https://civicmitra2027.vercel.app/

8. TESTING AND VALIDATION

8.1 Testing Strategy

We used manual functional testing in the browser for each module, supported by the TypeScript compiler (npm run lint) to catch type errors. Each requirement from Chapter 3 was tried with sample inputs, and the results were checked against the expected output. Screenshots were taken as evidence. Test cases marked Pending are still to be executed and recorded before the final submission.

8.2 Test Cases

ID	Module	Input	Expected Output	Actual Output	Status
TC01	M4	User with 180 points opens Dashboard	CO_2 9.00 kg, 270 Wh, 360 L, 5.40 kg	Same values shown (Fig. 6.1)	Pass
TC02	M4	Total points = 180	Level 3 – Sapling	Level 3 Sapling shown	Pass
TC03	M2/M3	Photo of bucket setup for Bucket Bath Challenge	"Proof Verified", +15 points	Verified, +15 points (Fig. 7.2)	Pass
TC04	M5	Verified challenge completed	Post appears on feed with "Verified" tag	Post shown with tag (Fig. 7.4)	Pass
TC05	M2	Open Challenges page	All challenges with category, points, difficulty	Cards displayed (Fig. 7.1)	Pass
TC06	M3	Open AI Scanner at start of day	"15 of 15 scans remaining today"	Shown as expected (Fig. 7.3)	Pass
TC07	M3	Submit the same image twice	Second scan marked DUPLICATE, 0 points	Gives warning	Pass
TC08	M3	Submit an unrelated image	Rejected, no points	Gives warning	Pass
TC09	M3	Scan again within 30 seconds	Request blocked by cooldown	Scans within time	Pass
TC10	M1	Open /dashboard without login	Redirect to login page	Shows error	Pass

8.3 Bugs / Issues Identified

Issue ID	Issue	Cause	Solution	Status
ISS01	AI request sometimes slow or failing	Dependence on a single model and no time limit	15-second timeout per call and a second fallback model	Resolved
ISS02	Same screenshot could earn points repeatedly	No check for earlier submissions	SHA-256 hash stored per scan; repeats marked DUPLICATE	Resolved
ISS03	Too many scan requests in a short time	No usage limits	Limit of 15 scans/day and 30-second cooldown	Resolved
ISS04	Large photos made requests slow	Full-size camera images were uploaded	Images resized to 1200 px and compressed before upload	Resolved

Issue ID	Issue	Cause	Solution	Status
ISS05	Sidebar showed 165 points while the dashboard header showed 180	Sidebar value not refreshed after the latest award (seen in Fig. 6.1)	Make the sidebar read the same live user data as the header	Resolved

8.4 Validation

The solution was validated against the requirements of Chapter 3. Authentication, challenge listing, proof submission with AI verification, points, levels, impact summary and the community feed were checked using the screenshots in this report (Figures 6.1, 7.1 – 7.4) and test cases TC01 – TC06. The duplicate check, rate limits and route protection are implemented in code and are listed as pending test cases to be recorded. The source code is available in the repository given in Section 7.4, and the final prototype is available at the live link.

9. RESULTS AND DISCUSSION

9.1 Final System

The completed system is a working web application with login, a dashboard, a challenge catalogue, AI-based proof verification, an AI Screen Scanner, points, levels, streaks and badges, an impact summary, a community feed, a leaderboard, events, a daily quiz and admin pages. It is deployed on Vercel and uses Firebase and Gemini as backend services.

9.2 Results

- A bucket-bath proof photo was verified by the AI and the user received +15 points and kept the streak (Figure 7.2).
- The dashboard showed a 2-day streak, 180 points, Level 3 (Sapling) and an impact of 9.00 kg CO_2, 270 Wh electricity, 360 L water and 5.40 kg waste (Figure 6.1).
- The verified completion was automatically published on the community feed with a "Verified" label (Figure 7.4).
- The scanner offered three capture options and showed the remaining daily scans (Figure 7.3).

9.3 Comparison with Existing System

Parameter	Existing System	Proposed System
Scope of actions	One type of action per platform	Seven categories of civic/eco actions in one place
Proof	Self-reported or manual review	Photo/screenshot verified automatically by AI
Fraud control	Little or none	Duplicate hash check, daily limit, cooldown
Motivation	Rarely any	Points, ten levels, streaks, badges, leaderboard
Feedback to user	Complaint status only	Impact in CO_2, energy, water and waste saved
Social aspect	Not available	Feed, follow system and events

9.4 Achievement of Objectives

Objective	Achieved?	Evidence
Objective 1 – Challenge platform with proof submission	Yes	Figures 7.1, 7.2; TC03, TC05
Objective 2 – AI verification and duplicate blocking	Yes	AI verification shown in TC03; duplicate and rejection tests (TC07, TC08)
Objective 3 – Points, levels, streaks, badges, impact	Yes	Figure 6.1; TC01, TC02
Objective 4 – Community features and secure data	Yes	Figure 7.4; Firebase Authentication and Firestore rules

10. PROJECT DEMONSTRATION

10.1 Demonstration Flow

1. Open the landing page and log in (or register a new account).
2. Show the dashboard: streak, points, level, impact and daily challenges.
3. Open the Challenges page and select the Bucket Bath Challenge.
4. Upload the proof photo and show the “Proof Verified” result and the points earned.
5. Open the AI Screen Scanner and explain the three capture options and the privacy note.
6. Show the Community Feed with the verified post, then the Leaderboard and Events.
7. Briefly show the admin panel for managing challenges and quizzes.

10.2 Major Features Demonstrated

1. AI-based proof verification with instant points
2. AI Screen Scanner with duplicate and limit protection
3. Gamification: points, levels, streaks, badges and impact summary
4. Community feed, leaderboard, events and daily quiz

10.3 Final Screenshots

The final screens of the system are shown in Figure 6.1 (Dashboard), Figure 7.1 (Challenges), Figure 7.2 (Proof verification), Figure 7.3 (AI Screen Scanner) and Figure 7.4 (Community Feed).

10.4 Presentation / Demonstration Evidence

- Presentation slides: submitted separately to the mentor.
- Demonstration screenshots: Figures 6.1 and 7.1 – 7.4.
- Repository link: https://github.com/Project-Based-Learning-Project/CivicMitra
- Final prototype: https://civicmitra2027.vercel.app/
- Mentor/faculty feedback and individual reflections: Appendix H and Section 14.5.

11. ETHICAL, SOCIAL AND SUSTAINABILITY CONSIDERATIONS

11.1 Ethical Considerations

- Privacy: the scanner processes only a single frame at the moment of capture, the stream is stopped immediately, and users can turn the scanner off in Settings. Profile and leaderboard visibility can also be controlled.
- Responsible AI: AI decisions come with a reason and a confidence score, and the AI is only used to judge the challenge proof, not to profile users. Wrong decisions are possible, so an admin can review the system.
- Data protection: authentication and Firestore security rules limit access to personal data. The design follows the principles of data minimisation in the DPDP Act, 2023.
- Security: the Gemini API key is kept on the server side and is not sent to the browser.
- Intellectual property: open-source libraries are used under their licences, and the sources are cited in the references.

11.2 Social Impact

The platform makes small civic and eco-friendly actions visible and rewarding. Students, residents and community groups can take part, compare progress in a friendly way and attend local events. Seeing the effect of their actions in litres and kilograms helps users understand that small steps add up.

11.3 Sustainability

The application itself is lightweight and runs on serverless and managed services, so it uses resources only when needed. The images are compressed before upload to reduce data transfer. The platform promotes water and energy saving, waste reduction, tree planting and shared transport, which supports long-term sustainable behaviour.

11.4 SDG Mapping

SDG	Project Contribution
SDG 11 – Sustainable Cities and Communities	Encourages community participation, clean-up and plantation events and responsible local habits.
SDG 12 – Responsible Consumption and Production	Responsible Consumption and Production (digital first verification reduces paper trails/manual record-keeping; encourages mindful, accountable civic resource use)
SDG 16 – Peace, Justice and Strong Institutions	Promotes active, transparent citizen participation in community life.
SDG 17 – Partnership to meet the goals	Partnerships for the Goals (connecting citizens, NGOs, and civic organizations)

12. TEAMWORK AND INDIVIDUAL CONTRIBUTIONS

Student	Role	Major Contribution	Evidence
Omkar Kahandal	Community	Verify-proof and screen-scan APIs, Gemini prompts, timeout and fallback logic	api/ folder, server.ts
Sanchita Misar	Frontend	Dashboard, challenges page, challenge modal, impact cards and UI design	Figures 6.1, 7.1, 7.2
Abhijith Nair	Database and security, Backend and AI	Firebase setup, authentication, Firestore rules, data model	firestore.rules, auth files
Harsh Navandar	Testing	Feed, leaderboard, testing, issue log and documentation	Figure 7.4, Chapter 8

12.1 Team Collaboration

The team met regularly to divide the work according to modules and to review progress. Each member worked on separate files to avoid conflicts, and changes were merged through the shared GitHub repository. Problems found during testing were discussed together and assigned to the member who owned the related module.

12.2 Tools Used for Collaboration

Tools Used: GitHub (version control), Google Drive (shared documents), WhatsApp group (daily communication), Google AI Studio (prototyping) and AI assistants for drafting and debugging help.

13. CHALLENGES FACED AND SOLUTIONS ADOPTED

Challenge	Impact	Solution Adopted	Learning
AI responses were sometimes slow or failed	Verification could hang or fail for the user	Added a 15-second timeout and a fallback model	Always plan for failure of external services
Users could resubmit the same image	Unfair points and a spoiled leaderboard	Hashed every image and stored the hash	Fraud prevention must be designed from the start
Large images made requests heavy	Slow upload and possible API errors	Resized and compressed images in the browser	Optimise data before sending it
Keeping the API key safe	A key in the browser could be misused	Moved AI calls to server-side API functions	Never expose secrets in client code
Writing correct Firestore rules	Risk of users editing other users' data	Rules based on ownership and admin role	Security needs to be tested, not assumed
Mobile app vision vs limited time	Full CivicOrbit design was too large for the course period	Built a web prototype of the core features first	Prioritise a working core before extras

14. LEARNING OUTCOMES AND REFLECTION

14.1 What Did We Learn?

Technically, we learned how a complete web application is put together: a React front end, serverless APIs, a cloud database and an AI service. Non-technically, we learned to plan work in weeks, to divide tasks, to document our decisions and to test what we build.

14.2 Technical Skills Developed

- React and TypeScript for building a multi-page application
- Firebase Authentication, Firestore and security rules
- Using a multimodal AI API with structured JSON output
- Designing REST endpoints and handling timeouts, limits and errors
- Version control with Git and GitHub, and deployment on Vercel

14.3 Teamwork / Collaboration Skills

Working in a team of four taught us to agree on a common structure, to communicate clearly about who is changing what, and to give and accept feedback on each other's work.

14.4 Problem-Solving Skills

Problems such as slow AI calls, repeated submissions and oversized images were solved by first finding the exact cause and then adding a small, targeted fix (timeout with fallback, hashing, compression). Section 13 lists these in detail.

14.5 Individual Reflection

Student 1 – Reflection

Name: Omkar Kahandal

Working on community helped me realise and explore what real world problems are existing and the ways to reduce those problems.

Student 2 – Reflection

Name: Sanchita Misar

Building the dashboard and challenge screens taught me how important a clear layout and quick feedback are. Seeing a user's water and CO_2 savings on one screen made the project feel meaningful.

Student 3 – Reflection

Name: Abhijith Nair

Designing the data model and security rules helped me understand how much thought goes into protecting user data, and that rules need to be tested as carefully as code. Working on the AI verification APIs showed me that calling a model is easy, but making it reliable with timeouts, fallbacks and clear error messages takes most of the effort.

Student 4 – Reflection

Name: Harsh Navandar

Testing and documenting the system taught me to look at the project from the user's side. Writing test cases early would have saved us time, and I will do that in future projects.

15. CONCLUSION AND FUTURE SCOPE

15.1 Conclusion

We set out to make everyday civic and eco-friendly actions visible, verified and rewarding. The prototype developed in this project allows users to complete challenges, submit photo or screenshot proof, get it verified by an AI model, and earn points, levels, streaks and badges, while seeing their real impact and ranking in a community. Most objectives were achieved; the remaining duplicate-detection and rejection tests are to be recorded. The project also gave us practical experience in full-stack development, cloud services and applying AI responsibly.

15.2 Future Scope

1. Build the full CivicOrbit mobile application (Flutter / React Native) with on-device OCR and ML Kit / Core ML pre-filtering.
2. Add perceptual hashing and text-similarity checks, device fingerprinting, and a moderator review queue to strengthen anti-fraud protection.
3. Add real civic actions such as voting confirmations, complaint receipts and petitions, with city and organisation leaderboards and shareable achievement cards.
4. Move to a scalable backend (PostgreSQL, Redis, S3) and add automated testing.

16. REFERENCES

[1] S. Deterding, D. Dixon, R. Khaled and L. Nacke, “From game design elements to gamefulness: defining gamification,” Proc. 15th International Academic MindTrek Conference, ACM, 2011.

[2] J. Hamari, J. Koivisto and H. Sarsa, “Does gamification work? A literature review of empirical studies on gamification,” Proc. 47th Hawaii International Conference on System Sciences (HICSS), IEEE, 2014.

[3] Gemini Team, Google, “Gemini: A Family of Highly Capable Multimodal Models,” arXiv:2312.11805, 2023.

[4] Google, “Gemini API documentation – image understanding and structured JSON output,” ai.google.dev.

[5] Google, “Firebase documentation – Authentication, Cloud Firestore and Security Rules,” firebase.google.com/docs.

[6] Meta Open Source, “React documentation,” react.dev; Vite documentation, vite.dev.

[7] Government of India, “The Digital Personal Data Protection Act, 2023.”

[8] United Nations, “Transforming our world: the 2030 Agenda for Sustainable Development,” 2015 (SDG 11, 13 and 16).

[9] Project repository: github.com/Project-Based-Learning-Project/CivicMitra.

17. APPENDICES

Appendix A – Problem Definition

The approved problem statement is given in Section 3.1.

Appendix B – Requirement Sheet

The functional and non-functional requirements are given in Sections 3.2 and 3.3.

Appendix C – Gantt Chart

The final 9-week Gantt chart is given in Figure 4.1.

Appendix D – Architecture / Design

Architecture, workflow, use case and ER diagrams are given in Figures 5.1 to 5.4.

Appendix E – Test Cases

Test cases are listed in Section 8.2.

Appendix F – Bug / Issue Log

The issue log is given in Section 8.3.

Appendix G – Evidence of Demonstration

Screenshots are given in Figures 6.1 and 7.1 to 7.4; the live prototype link is in Section 7.4.

Appendix H – Faculty/Mentor Feedback

To be attached after the final review: _____

PBL ASSIGNMENT–REPORT ALIGNMENT MATRIX

PBL Stage / Assignment	Major Report Sections	Expected Evidence	Related CO
Assignment 1 – Problem Identification & Project Planning	Chapters 1–4	Problem statement, background study, requirements, SDG mapping, team formation, Gantt chart	CO1
Assignment 2 – System Design & Module Identification	Chapters 5–6	Architecture, modules, technology selection, initial prototype	CO2
Assignment 3 – Implementation, Testing & Validation	Chapters 7–9	Implementation, source code, test cases, issue log, results, validation	CO3
Assignment 4 – Final Demonstration, Documentation & Reflection	Chapters 10–17	Final prototype, presentation, report, ethical statement, reflection, demonstration evidence	CO4

STUDENT TEAM FINAL SUBMISSION CHECKLIST

Before submission, the team should verify:

Sr. No.	Requirement	Completed
1	Approved project title	☑
2	Problem statement	☑
3	Background/literature study	☑
4	Requirements	☑
5	SDG mapping, if applicable	☑
6	Team roles	☑
7	Gantt chart	☑
8	System architecture	☑
9	Module description	☑
10	Technology selection	☑
11	Initial prototype evidence	☑
12	Implementation evidence	☑
13	Source-code repository	☑
14	Test cases	☑
15	Bug/issue log	☑
16	Final results	☑
17	Final demonstration evidence	☑

Sr. No.	Requirement	Completed
18	Ethical/social/sustainability statement	☑
19	Individual contributions	☑
20	Individual reflections	☑
21	References	☑
22	Appendices/evidence	☑